

المحاضرة السابعة: تصميم الهيكل الأساسي للتطبيق_ أدوات التنقل والقوائم في Flutter

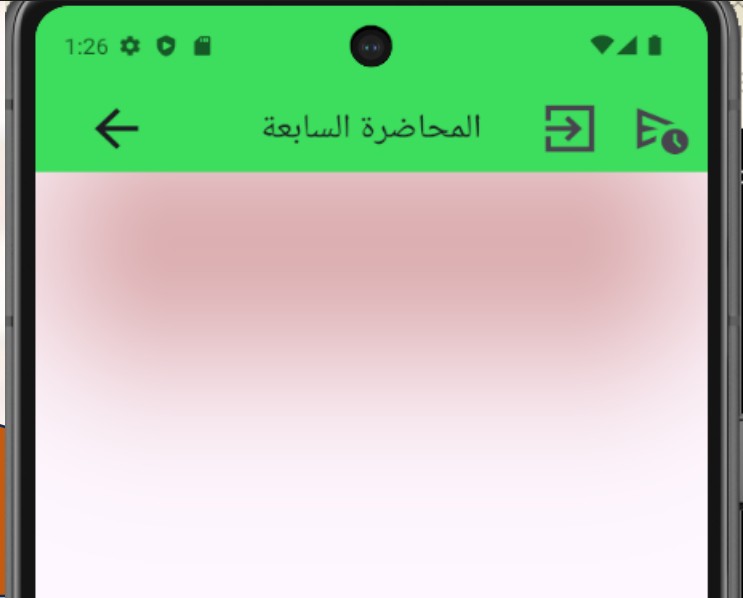


م. سليمان الشوصي

❖ محتويات المحاضرة

- **AppBar**: اهم خصائص شريط التطبيق العلوي (العنوان، الأيقونات، والإجراءات).
- **ListTile**: هيكلية وتنسيق عناصر القوائم (النصوص، والأيقونات المدمجة).
- **Drawer**: تصميم القائمة الجانبية للتنقل وترويسة حساب المستخدم (UserAccountsDrawerHeader).
- **CircleAvatar**: عرض الصور والأيقونات بقالب دائري (تُستخدم لصور الملف الشخصي).
- **TabBar & TabBarView**: بناء واجهات التبويبات المتزامنة أفقياً.
- **TabController**: إدارة حالة التبويبات (عبر المتحكم الافتراضي DefaultTabController أو المتحكم المخصص).
- **BottomNavigationBar**: برمجة شريط التنقل السفلي وإدارة تبديل الشاشات.

```
return Scaffold(  
  appBar: AppBar(  
    title: Text("المحاضرة السابعة"),  
    centerTitle: true,  
    backgroundColor: const Color.fromARGB(255,54,244,86),  
    elevation: 100,shadowColor: Colors.red,  
    leading: IconButton( onPressed: () {},  
      icon: Icon(Icons.arrow_back, size: 40),), // IconButt  
    leadingWidth: 100,  
    actions: [  
      IconButton(onPressed: () {},  
        icon: Icon(Icons.exit_to_app_sharp, size:40),), //  
      IconButton(onPressed: () {},  
        icon:Icon(Icons.schedule_send_outlined,size:40),) /  
    ]), // AppBar  
  );}
```



الأداة ال AppBar: ، وهي شريط التطبيق العلوي الذي يظهر في أعلى الشاشة لتسهيل التنقل والتفاعل.

الخصائص الأساسية والإضافية.

الخاصية	عملها
title	حددت عنوان شريط التطبيق بالنص "المحاضرة السابعة".
centerTitle	أعطيت القيمة true لتوسيط النص في منتصف الشريط.
leading	تضع عنصرًا في بداية الشريط (الجهة اليسرى)، وهنا أضيف IconButton يحمل أيقونة السهم للرجوع Icons.arrow_back بحجم ٤٠.
leadingWidth	تُحدد عرض المساحة المخصصة لعنصر ال leading (تمت زيادتها إلى ١٠٠).
actions	تستقبل قائمة List من الأدوات لتوضع في نهاية الشريط (الجهة اليمنى)، وتضم هنا زرین لأيقونات التفاعل الخروج والجدولة (exit_to_app_sharp و schedule_send_outlined).

```

ListTile(
  title: Text("home_page"),
  tileColor: const Color.
    fromARGB(255, 173, 208, 174),
  subtitle: Text("1"),
  leading: Icon(Icons.home),
  trailing: Text("main_home"),
  onTap: () {},
  onLongPress: () {
    print("Longpress");
  },
), // ListTile

```

الأداة الـ ListTile: الـ ListTile هو Widget مصمم خصيصاً لتنظيم المحتوى في صف واحد يحتوي على مساحات محددة مسبقاً للصور، النصوص الأساسية، والنصوص الفرعية، وأيقونات الإجراءات.

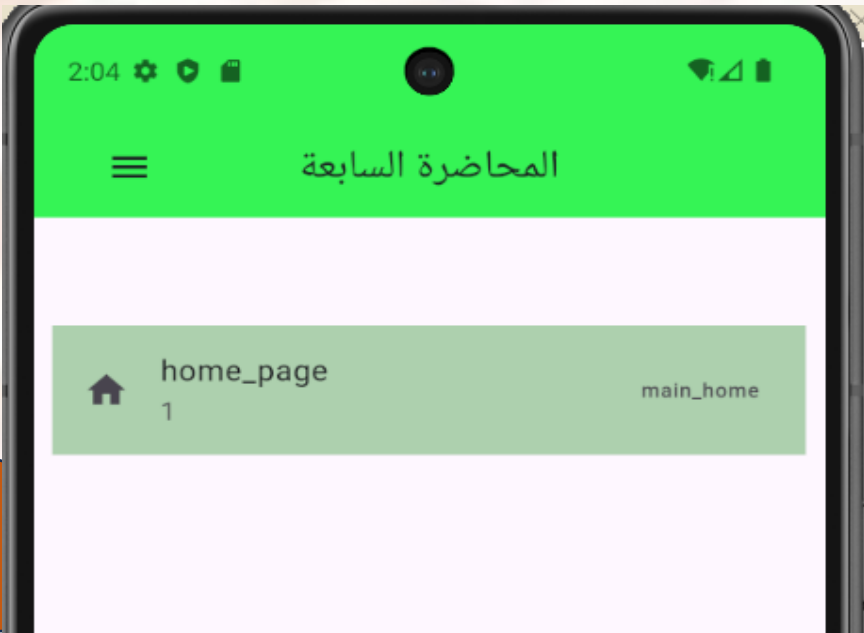
الخصائص الأساسية للمحتوى

❑ **title:** يمثل العنصر النصي الأساسي لصف القائمة، عادةً ما يُستخدم لعرض اسم العنصر أو عنوانه الرئيسي، ويظهر بخط أكبر وأكثر بروزاً من النص الفرعي، يتم تمرير **Text Widget** إليه.

❑ **subtitle:** يمثل العنصر النصي الثانوي أو الوصفي، يظهر عادةً بخط أصغر وأقل بروزاً أسفل الـ **title**. يُستخدم لتوفير تفاصيل إضافية أو وصف موجز يتم تمرير **Text Widget** إليه.

❑ **leading:** تحديد الـ Widget الذي يظهر في **الجهة البادئة** (على اليمين في اتجاه RTL كالعربية)، يُستخدم عادةً لعرض أيقونة (Icon)، صورة (CircleAvatar أو Image)، أو صورة مصغرة.

❑ **trailing:** تحديد الـ Widget الذي يظهر في **الجهة اللاحقة** أو النهائية (على اليسار في اتجاه RTL). يُستخدم عادةً لأيقونات الإجراءات (مثل أسهم التنقل أو أزرار التبديل Switch).



```
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(backgroundColor: Theme.of(context).colorScheme.inverse
      leading: IconButton(onPressed: () {}, icon: Icon(Icons.arrow_back)
      title: Text(widget.title, style: TextStyle(fontSize: 18)),
    ), // AppBar
    // drawer: ,
    endDrawer: Drawer(
      child: Column( children: [
        UserAccountsDrawerHeader(
          currentAccountPicture: CircleAvatar(
            backgroundColor: const Color.fromARGB(255, 25, 23, 23),
            child: Icon(size: 50, Icons.person_add_alt,
              color: Colors.white)), // Icon // CircleAvatar
          accountName: Text("suliman"),
          accountEmail: Text("sulimanalshossi@gmail.com"),
        ), // UserAccountsDrawerHeader
```

```
endDrawer: Drawer(
  child: Column( children: [
    Container(width: double.infinity, height: 200,
      padding: EdgeInsets.all(50),
      color: const Color.fromARGB(255, 62, 82, 101),
      child: Column(children: [
        Icon(size: 50, Icons.person_add_alt, color: Colors.white),
        Text( "الحساب الشخصي",
          style: TextStyle(color: Colors.white, fontSize: 18.0),
        ), // Text
      ]), // Column // Container
```

❖ **الـ Widgets المتقدمة للهيكل والقوائم الجانبية:** تستخدم بالعادة داخل الـ Scaffold لتقديم قوائم تنقل أو إعدادات بشكل جانبي.

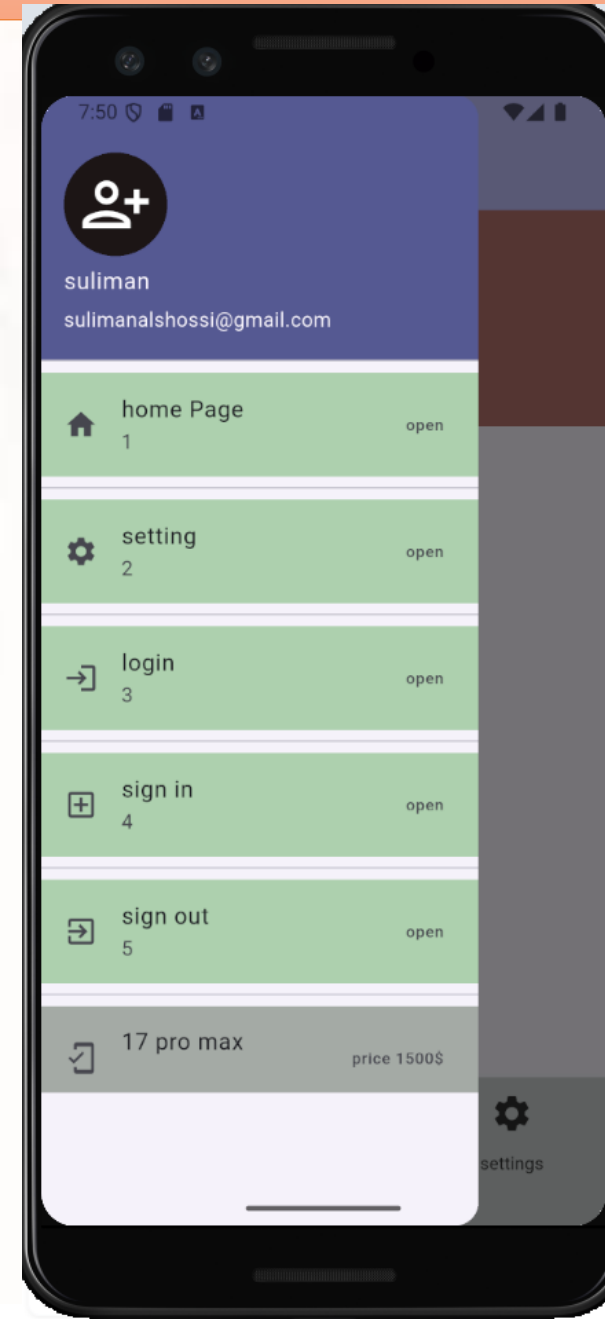
❖ **Drawer:** تستقبل Widget واحدًا (وهو Drawer) حيث تمثل القائمة الجانبية الأساسية، التي تنزلق من الجانب الأيمن الى اليسر، تستخدم في حال كان التطبيق الإنجليزية. وظيفته الأساسية هي توفير خلفية بيضاء ومساحة عرض قياسية لأي محتوى تنقل.

• **UserAccountsDrawerHeader:** راس القائمة: هذا الـ Widget هو قالب مُحسّن ومُخصص لعرض معلومات أساسية عن المستخدم في الجزء العلوي من القائمة الجانبية.

أهم الخصائص:

- **currentAccountPicture:** تُخصص لعرض صورة أو رمز يمثل الحساب الحالي. يتم وضع Widget داخلها، ويكون الـ CircleAvatar هو الخيار الأنسب لتقديم صورة دائرية.
 - **accountName:** تعرض اسم المستخدم الرئيسي بخط مُكبر وأسلوب مميز. يتم تمرير Text Widget يحمل اسم المستخدم إليها.
 - **accountEmail:** تعرض معلومات إضافية للحساب، غالباً ما تكون البريد الإلكتروني، أسفل الاسم. يتم تمرير Text Widget يحمل البريد الإلكتروني إليها.
 - **CircleAvatar الصورة الدائرية:** Widget متخصص لتقديم المحتوى داخل حاوية دائرية الشكل.
- ❖ **endDrawer:** تمثل القائمة الجانبية الثانوية أو المعاكسة، التي تنزلق من الجانب اليسر الى الجانب الأيمن، تستخدم في حال كان التطبيق باللغة العربية.

```
drawer: Drawer(  
  child: Column( children: [UserAccountsDrawerHeader(  
    currentAccountPicture: CircleAvatar(  
      backgroundColor: const Color.fromARGB(255, 25, 23, 23),  
      child: Icon( size: 50,Icons.person_add_alt,color: Colors.white,),  
    ), // CircleAvatar  
    accountName: Text("suliman"),  
    accountEmail: Text("sulimanalshossi@gmail.com"),  
  ), // UserAccountsDrawerHeader  
  ListTile(subtitle: Text("1"),title: Text("home Page"),  
    leading: Icon(Icons.home),trailing: Text("open"),  
    tileColor: const Color.fromARGB(255, 173, 208, 174),  
    onTap: () {}, onLongPress: () {}, ), // ListTile  
  Divider(),  
  ListTile( subtitle: Text("2"),title: Text("setting"),  
    leading: Icon(Icons.settings),trailing: Text("open"),  
    tileColor: const Color.fromARGB(255, 173, 208, 174),  
    onTap: () { }, onLongPress: () {}), // ListTile  
  Divider(),  
  ListTile( subtitle: Text("3"),title: Text("login"),  
    leading: Icon(Icons.login), trailing: Text("open"),  
    tileColor: const Color.fromARGB(255, 173, 208, 174),  
    onTap: () { }, onLongPress: () {}), // ListTile  
  Divider(),  
  ListTile( subtitle: Text("4"),title: Text("sign in"),  
    leading: Icon(Icons.add_box_outlined), trailing: Text("open"),  
    tileColor: const Color.fromARGB(255, 173, 208, 174),  
    onTap: () { }, onLongPress: () {}), // ListTile  
  Divider(),  
  ListTile(subtitle: Text("5"),title: Text("sign out"),  
    leading: Icon(Icons.exit_to_app),trailing: Text("open"),  
    tileColor: const Color.fromARGB(255, 173, 208, 174),  
    onTap: () { }, onLongPress: () {}), // ListTile
```



```
class Lesson3 extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return DefaultTabController(
      length: 2,
      child: Scaffold(
        appBar: AppBar(
          title: Text("المحاضرة السابعة TapBarview"),
          backgroundColor:
            const Color.fromARGB(255, 54, 244, 86),
          centerTitle: true, // AppBar
        ),
        body: TabBarView(
          children: [
            Text("Home Page1"),
            Text("Home Page2")
          ], // TabBarView
        ), // Scaffold // DefaultTabController
      );
    }
  }
}
```

❖ **أداة DefaultTabController:** هي أداة (Widget) تُستخدم لتوفير وإدارة متحكم تبويبات (TabController) بشكل افتراضي للعناصر الفرعية المرتبطة بها (مثل TabBar و TabBarView)، وذلك دون الحاجة إلى تعريف متحكم مخصص وإدارة دورة حياته برمجياً بشكل يدوي.

أهم ميزاتها وخصائصها:

- **الإدارة التلقائية للذاكرة:** تتولى الأداة مهمة إنشاء المتحكم وتدميره (Dispose) من الذاكرة تلقائياً عند عدم الحاجة إليه، مما يبسط كتابة الكود ويمنع تسرب الذاكرة.
- **خاصية الطول (length):** هي خاصية إلزامية يجب تمريرها للأداة، ووظيفتها تحديد العدد الإجمالي لعلامات التبويب (Tabs) التي سيتم عرضها.
- **آلية العمل:** تُوضع كعنصر "أب" (Parent) يغلف الواجهة بالكامل (يغلف عادةً أداة Scaffold).

بفضل ذلك، يستطيع كل من TabBar و TabBarView العثور على هذا المتحكم الافتراضي في شجرة الأدوات (Widget Tree) والتزامن مع بعضهما البعض تلقائياً بناءً على مؤشر التبويب النشط.

❖ **أداة TabBarView:** تُعد أداة TabBarView مكوّناً هيكلياً أساسياً لبرمجة واجهات المستخدم التبويبية في فلاتر. تعمل هذه الأداة كعارض صفحات أفقي (Horizontal View Pager)، وتكمن وظيفتها المحورية في تجسيد المحتوى المرئي لكل تبويب (Tab). تضمن الأداة الحفاظ على تزامن دقيق بين مؤشر التبويب النشط والشاشة المعروضة للمستخدم.

آلية العمل والاعتمادية

- **الارتباط بمتحكم (TabController):** لا تعمل الأداة بشكل مستقل، بل تتطلب وجود متحكم مشترك (TabController) يربطها بشريط التبويبات (TabBar). يتولى هذا المتحكم إدارة مؤشر التبويب الحالي (Index)، وبمجرد تغير هذا المؤشر، يقوم TabBarView بتبديل الواجهات (Widgets) لعرض الشاشة المطابقة.
- **التنقل عبر الإيماءات (Gestural Navigation):** تدعم الأداة الانتقال بين الشاشات افتراضياً عبر إيماءات السحب الأفقي (Swiping). عند قيام المستخدم بسحب الشاشة، يتم تحديث قيمة المتحكم برمجياً، لينعكس ذلك فوراً على تظليل التبويب الجديد في TabBar، مما يضمن تجربة استخدام سلسة ومتكاملة.

الخصائص الوظيفية الرئيسية

- **خاصية children:**
 - **الوظيفة:** خاصية إجبارية تستقبل قائمة (List) من الواجهات (Widgets) التي تُمثل محتوى كل صفحة.
 - **الشرط التقني:** يجب أن يتطابق عدد العناصر المدرجة في هذه القائمة تماماً مع عدد علامات التبويب الموجودة في TabBar. يتم ربط كل عنصر بتبويبه المطابق استناداً إلى ترتيب المؤشر (Index).
- **خاصية controller:**
 - **الوظيفة:** تُستخدم لتمرير كائن من نوع TabController لتأمين الاتصال والمزامنة مع شريط التبويبات.
 - **الآلية الافتراضية:** في حال عدم تحديد هذه الخاصية صراحةً، ستقوم الأداة تلقائياً بالبحث في هيكل الشجرة (Widget Tree) عن أداة DefaultTabController الأب للاعتماد عليها.


```
class _MyHomePageState extends State<MyHomePage> {
  @override
  Widget build(BuildContext context) {
    return DefaultTabController(
      length: 3,
      child: Scaffold(
        appBar: AppBar(
          backgroundColor: Theme.of(context).colorScheme.inversePrimary,
          title: Center(
            child: Text(widget.title, style: TextStyle(fontSize: 16)),
          ), // Center
        ), // AppBar
        body: TabBarView(
          children: [
            Container(width: double.infinity,
              color: const Color.fromARGB(255, 129, 122, 121),
              child: Text("Tap_1", style: TextStyle(fontSize: 20)),
            ), // Container
            Container( width: double.infinity,
              color: const Color.fromARGB(255, 54, 241, 244),
              child: Text("Tap_2", style: TextStyle(fontSize: 20)),
            ), // Container
            Container(width: double.infinity,
              color: const Color.fromARGB(255, 91, 104, 243),
              child: Text("Tap_3", style: TextStyle(fontSize: 20)),
            ), // Container
          ],
        ), // TabBarView
      ), // Scaffold
    ); // DefaultTabController
  }
}
```

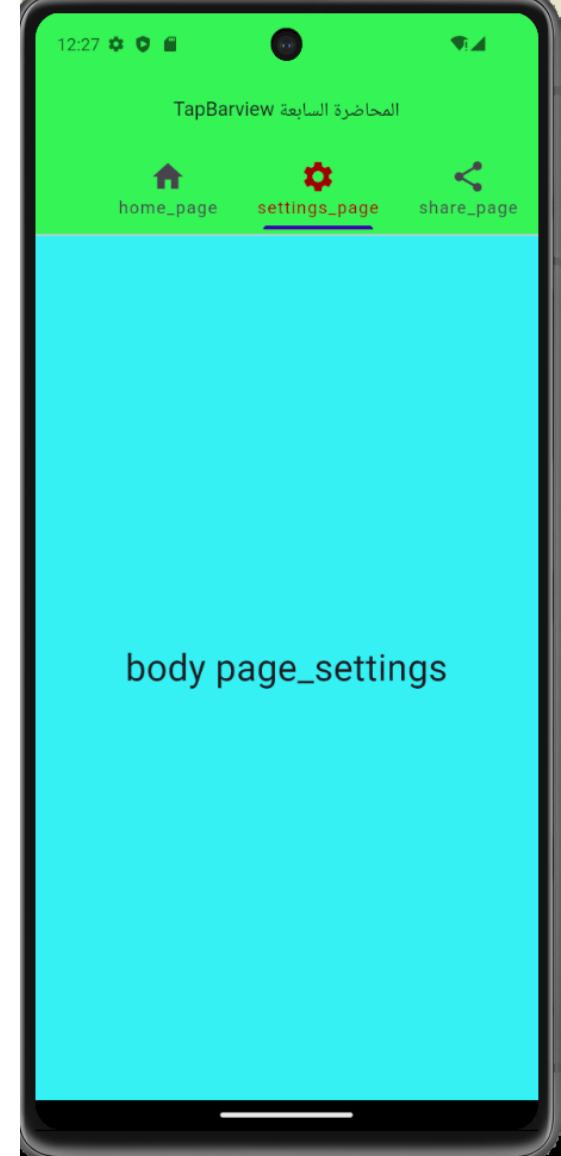
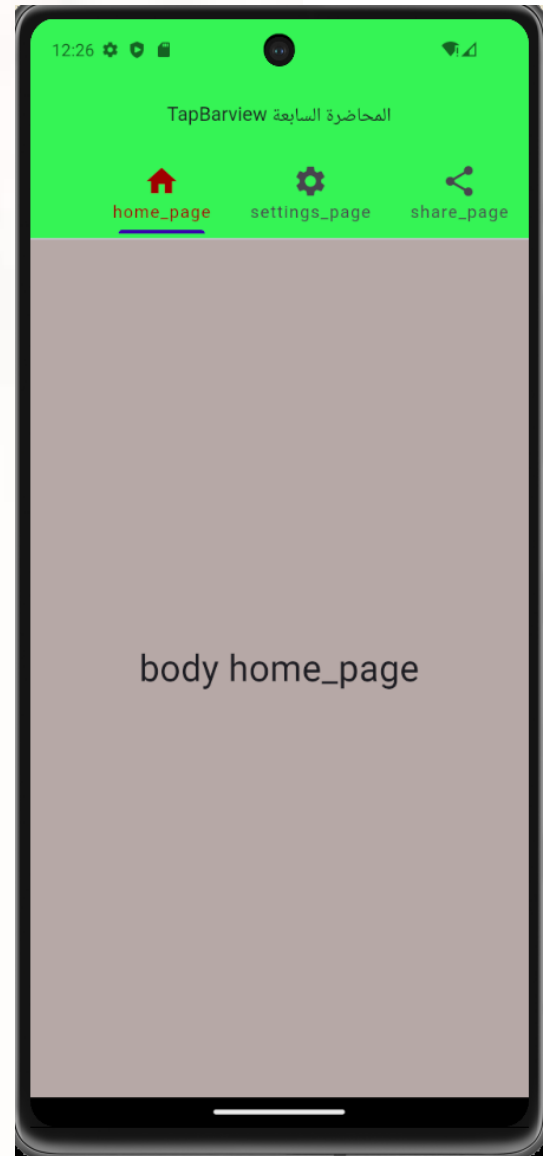
❖ **أداة TabBar (شريط التبويبات):** هو ويدجت (Widget) يمثل حاوية أفقية لعرض مجموعة من علامات التبويبات القابلة للتفاعل. يعمل كأداة تحكم مرئية ترتبط برمجياً بمتحكم (TabController) لإدارة الحالة ومزامنة التبويبات النشط مع واجهة العرض المقابلة له (TabBarView). يُوضع غالباً في الخاصية bottom الخاصة بـ AppBar.

خاصية tabs:

- **tabs (قائمة التبويبات):** هي خاصية (Property) إجبارية تابعة لأداة TabBar. تستقبل هيكل بيانات من نوع مصفوفة (List of Widgets)، وظيفتها تحديد عدد ومحتوى العناصر التي سيتم رسمها داخل الشريط. (برمجياً: يجب أن يتطابق عدد العناصر في هذه المصفوفة مع الطول length المحدد في DefaultTabController).
- **isScrollable:** عند تعيينها إلى true، تسمح لشريط التبويبات بالتمرير أفقياً. تُستخدم عندما يكون عدد التبويبات كبيراً ولا يتسع لها عرض الشاشة بالكامل.
- **indicatorColor:** تُحدد لون المؤشر (وهو عادةً الخط السفلي) الذي يظهر تحت علامة التبويبات النشطة.
- **indicatorWeight:** تتحكم في سُمك (ارتفاع) خط المؤشر السفلي للتبويبات النشط.
- **indicatorPadding:** تُضيف هوامش فراغية (حشوة) حول خط المؤشر.
- **labelColor:** تُحدد لون النص أو الأيقونة لعلامة التبويبات النشطة فقط لتمييزها عن بقية التبويبات.
- **labelStyle:** تُستخدم لتطبيق تنسيقات مخصصة على نص التبويبات (مثل حجم الخط، أو نوعه). حيث يتم استخدام letterSpacing: 1 لإضافة تباعد بسيط بين أحرف النص.
- **onTap:** هي دالة استدعاء (Callback) تُنفذ فور النقر على أي تبويب. تمرر هذه الدالة رقم الفهرس (Index) الخاص بالتبويب المنقور عليه لتمكن من اتخاذ إجراء برمجي معين .

❖ **الأداة Tab (علامة التبويبات):** هو Widget يمثل عنصراً (Node) مفرداً داخل مصفوفة tabs. يُستخدم لبناء الواجهة البصرية للتبويبات الواحد، ويُخصص لعرض محتوى توضيحي للمستخدم يتكون عادةً من أيقونة (icon)، أو نص دلالي (child / text)، أو كليهما معاً.

```
class Lesson3 extends StatelessWidget {  
  @override  
  Widget build(BuildContext context) {  
    return DefaultTabController( length: 3,  
      child: Scaffold(appBar: AppBar(  
        backgroundColor: const Color.fromARGB(255,54,244,86),  
        title: Center(child: Text("TapBarview المحاضرة السابعة",  
          style: TextStyle(fontSize: 16))), // Center // Text // Center  
        bottom: TabBar(isScrollable: true,  
          indicatorColor: const Color.fromARGB(255, 55, 5, 173),  
          indicatorWeight: 3, indicatorPadding: EdgeInsets.all(5),  
          labelColor: const Color.fromARGB(255, 159, 5, 5),  
          labelStyle: TextStyle(letterSpacing: 1),  
          onTap: (value) { print(value);},  
          tabs: [  
            Tab(icon: Icon(Icons.home, size: 30), child: Text("home_page")),  
            Tab( icon: Icon(Icons.settings, size: 30),child: Text("settings_page")),  
            Tab(icon: Icon(Icons.share_sharp, size: 30),child: Text("share_page"),)], //  
        body: TabBarView( children: [  
          Container(width:double.infinity,color: const Color.fromARGB(255,182,168,166),  
            child:Center(child:Text("body home_page",style:TextStyle(fontSize:30))), // //  
          Container(width:double.infinity,color: const Color.fromARGB(255,54,241,244),  
            child:Center(child:Text("body page_settings",style:TextStyle(fontSize:30),)),  
          Container( width:double.infinity,color: const Color.fromARGB(255,187,141,5),  
            child:Center(child:Text("body share_page_3",style:TextStyle(fontSize:30),))),  
        ],),),}} // Scaffold // DefaultTabController
```



```
class Lesson3 extends StatefulWidget {
  Lesson3({super.key});
  @override
  TabBarPage createState() => _TabBarPage();
}

class _TabBarPage extends State<Lesson3>
    with SingleTickerProviderStateMixin {
  TabController? myTabController;

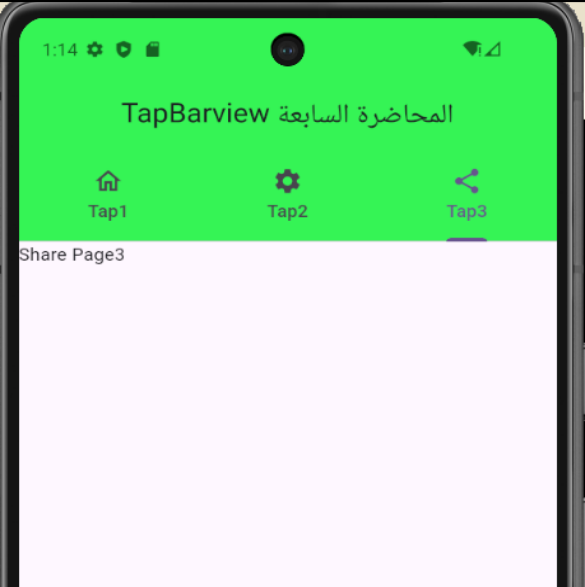
  @override
  void initState() {
    myTabController = new TabController(length: 3,
    vsync: this, initialIndex: 2,);
    super.initState();
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text("TapBarview السابعة"),
        backgroundColor: const Color.fromARGB(255, 54, 244, 86),
        centerTitle: true,
        bottom:
          TabBar(controller: myTabController,
            tabs: [
              Tab(icon: Icon(Icons.home_outlined),child:Text("Tap1")),
              Tab(icon: Icon(Icons.settings),      child:Text("Tap2")),
              Tab(icon: Icon(Icons.share),          child:Text("Tap3")),
            ]), // TabBar // AppBar
      body: TabBarView(controller: myTabController,
        children: [
          Text("Home Page1"),
          Text("Settings Page2"),
          Text("Share Page3"),
        ]), // TabBarView // Scaffold
    );
  }
}
```

الطريقة الثانية (اليدوية/المخصصة) توضح الفرق بين الطريقتين المستخدمتين لربط الـ TabBar مع الـ

Flutter:فيTabBarView

وجه المقارنة	الطريقة الأولى (التلقائية)	الطريقة الثانية (اليدوية/المخصصة)
المتحكم (Controller)	استخدام DefaultTabController لربط الشاشات تلقائياً.	إنشاء TabController يدوي داخل initState.
التأثيرات (Ticker)	لا تحتاج لـ Mixin.	تتطلب إضافة SingleTickerProviderStateMixin.
التحكم والتمرير	بسيطة: لا تسمح بالتحكم البرمجي المتقدم (مثل التبديل التلقائي من زر خارجي).	مرنة: تسمح بالتحكم الكامل، مثل تحديد التبويب الافتراضي (initialIndex: 2) أو الاستماع للتغييرات.
ربط اليدوي	يتم ربط الـ TabBar والـ TabBarView تلقائياً عبر الشجرة (Context).	يجب تمرير controller: myTabController يدوياً لكل من TabBar و TabBarView.



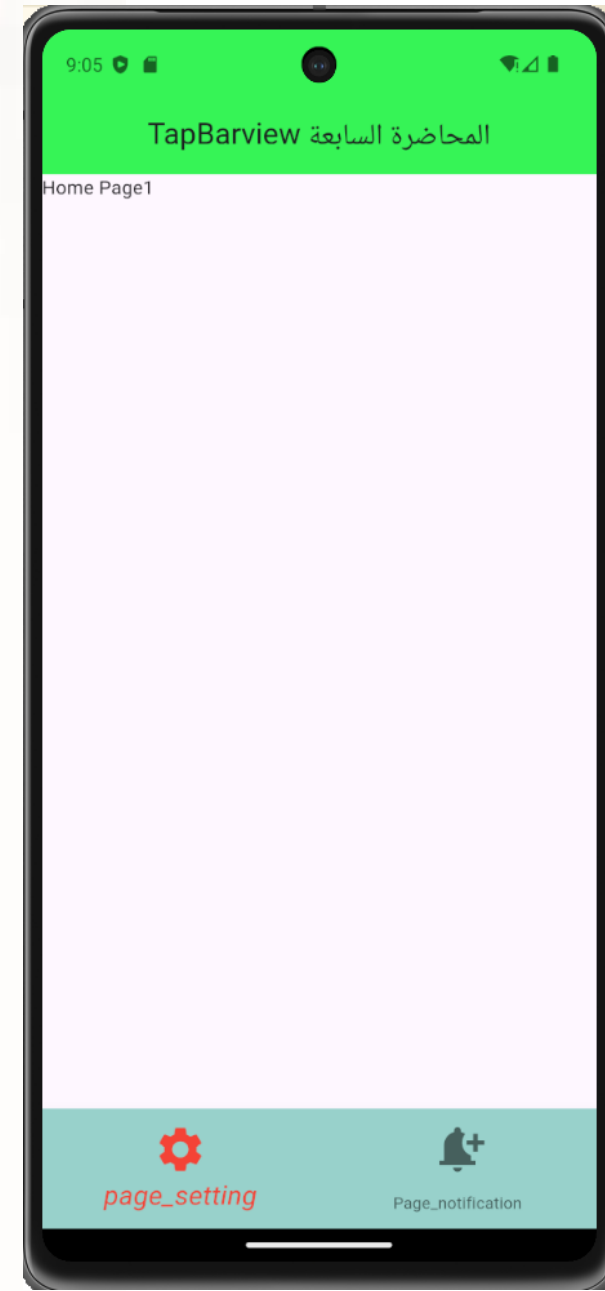
أداة **BottomNavigationBar Widget**: يعد أساسيًا لتمكين المستخدمين من التنقل السريع بين عدد محدود من الواجهات الرئيسية في التطبيق، حيث ان الـ BottomNavigationBar هو صف أفقي يظهر في أسفل شاشة Scaffold، ويسمح بالتبديل بين شاشات المستوى الأعلى (Top-level Screens) في التطبيق.

أهم الخصائص: هي التالي:

- **items (List<BottomNavigationBarItem>)**: هذه الخاصية إلزامية وتحدد العناصر التي ستظهر في شريط التنقل. كل عنصر هو Widget من نوع BottomNavigationBarItem. أيضا يجب أن يحتوي على **عنصرين على الأقل** ولا يُنصح أن يتجاوز **خمسة عناصر** للحفاظ على قابلية الاستخدام الجيدة.
- **currentIndex (int)**: وظيفته تحدد **مؤشر (Index)** العنصر الحالي النشط في شريط التنقل. يجب ربط هذه القيمة بمتغير حالة (State) ضمن StatefulWidget ليتمكن فلاتر من تمييز العنصر المختار حاليًا بصريًا.
- **onTap (ValueChanged<int>)**: هذه هي دالة وظيفتها رد الاتصال التي تُستدعى عندما يضغط المستخدم على أيقونة. تستقبل هذه الدالة **مؤشر (Index)** العنصر الذي تم النقر عليه، حيث تُستخدم هذه الدالة لتحديث قيمة **currentIndex** باستخدام **setState()**، ولتغيير محتوى الشاشة الـ body في Scaffold ليتوافق مع العنصر المختار.
- **type (BottomNavigationBarType)**: تُحدد سلوك ومظهر شريط التنقل عند وجود أكثر من ثلاثة عناصر.
 - **fixed**: ثابت: الحجم ثابت، وتظهر الأيقونات والعناوين بشكل دائم.
 - **shifting**: متحول: عند النقر على عنصر، يتوسع قليلاً، بينما تختفي عناوين العناصر غير المختارة.
- **selectedItemColor (Color)**: خاصية وظيفتها تحدد لون الأيقونة والنص للعنصر المختار حالياً.
- **unselectedItemColor (Color)**: خاصية وظيفتها تحدد لون الأيقونات والنصوص للعناصر غير المختارة.
- **خصائص العنصر الداخلي BottomNavigationBarItem**: كل عنصر في قائمة **items** يجب أن يكون من هذا النوع، وله خاصيتان رئيسيتان وهما:
 - **icon (Widget)**: هو الـ Widget الذي يمثل الأيقونة التي ستظهر للعنصر (عادةً Icon Widget).
 - **label (String)**: هو النص الذي يظهر أسفل الأيقونة. هذه الخاصية مطلوبة وتُستخدم كعنوان قصير للعنصر.

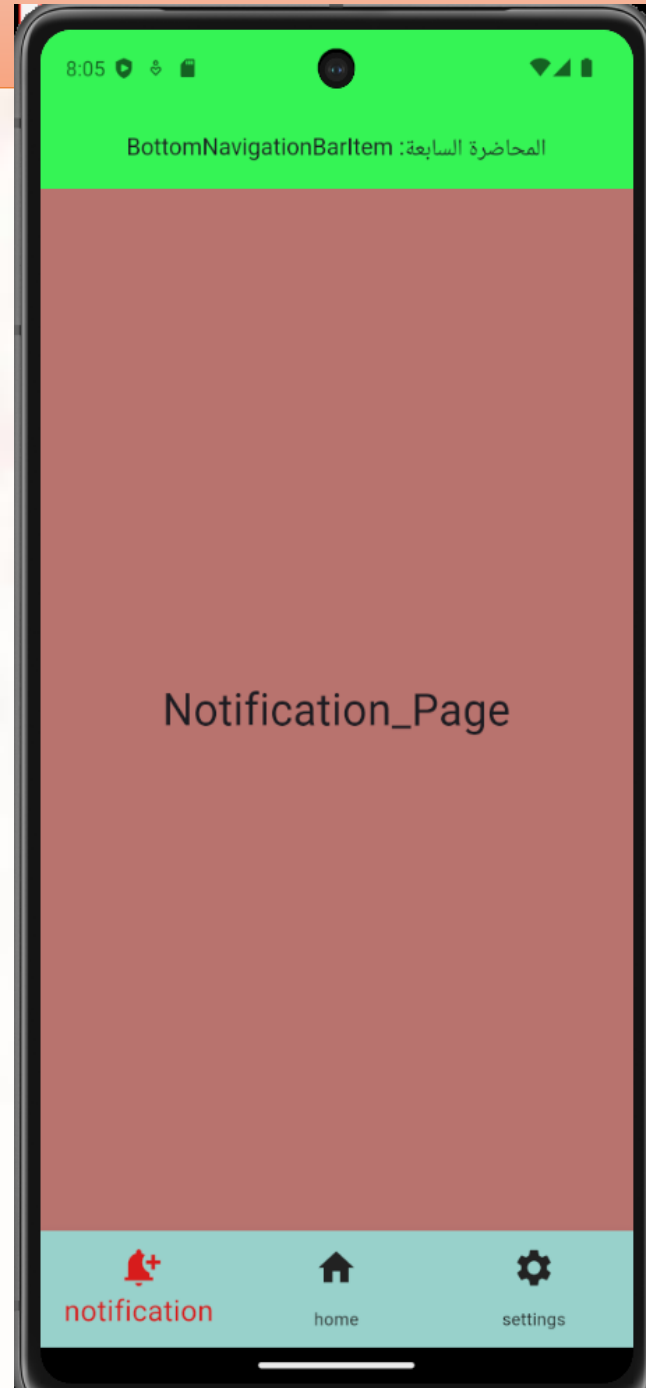
```
class _TapBarPage extends State<Lesson3> {
  var selectedIndex=0;

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar( // AppBar ...
        bottomNavigationBar: BottomNavigationBar(
          currentIndex: selectedIndex,
          selectedItemColor: Colors.red,
          selectedFontSize: 20,
          iconSize: 40,
          backgroundColor: const Color.fromARGB(255, 152, 209, 203),
          onTap: (index) {
            setState(() {selectedIndex=index; });
            print(index); },
          items:[
            BottomNavigationBarItem(
              icon: Icon(Icons.settings),
              label: "page_setting"), // BottomNavigationBarItem
            BottomNavigationBarItem(
              icon: Icon(Icons.notification_add),
              label: "Page_notification"])), // BottomNavigationBarItem
      body: Text("Home Page1"),);} // Scaffold
```




```
class _TapBarPage extends State<Lesson3> {
  int selectedIndex = 0;
  List<Widget> widgetPagesNavigationbottom = [
    Container(height:double.infinity,width:double.infinity,
      color: const Color.fromARGB(255, 184, 115, 110),
      child: Center(child: Text("Notification_Page",
        style:TextStyle(fontSize:30,))), // Text // Center
    Container(height:double.infinity,width:double.infinity,
      color: const Color.fromARGB(255,105,221,128),
      child:Center(child:Text("Home_Page",
        style:TextStyle(fontSize:30))), // Text // Center //
    Container( margin: EdgeInsets.all(10), height: 300,
      width: double.infinity,color: Colors.blue,
      child:Center(child:Text("Settings_Page",
        style:TextStyle(fontSize:30))), ),]; // Text // Cent
  @override
```

```
Widget build(BuildContext context) {
  return Scaffold(
    appBar:AppBar(
      title:Text("BottomNavigationBarItem: السابعة", //
    bottomNavigationBar:BottomNavigationBar(
      backgroundColor: const Color.fromARGB(255,152,209,203),
      currentIndex: selectedIndex, iconSize: 30, //0
      selectedItemColor: Color.fromARGB(255,215,28,28),
      selectedLabelStyle: TextStyle(fontSize: 20),
      unselectedItemColor: const Color.fromARGB(255,32,33,32),
      onTap: (index) {setState((){selectedIndex = index;}); },
      items: [ BottomNavigationBarItem(label:"notification",
        icon:Icon(Icons.notification_add),), // Bottom Navigatio
        BottomNavigationBarItem(
          label:"home",icon:Icon(Icons.home)), // Bottom Navigatio
        BottomNavigationBarItem(
          label: "settings",icon:Icon(Icons.settings),), //
      body:widgetPagesNavigationbottom.elementAt(selectedIndex));
```



إدارة التنقل القائمة على الفهرس والمصفوفة (Indexed List State Pattern) للتنقل بين الشاشات، وتتخصص آلية العمل في النقاط التقنية التالية:

- **تجميع الشاشات (List<Widget>):** يتم تعريف مصفوفة تحتوي على كافة الشاشات المراد التنقل بينها، حيث تُرتَّب الواجهات تسلسلياً برقم دليل (Index) يبدأ من ٠.
- **تتبع الحالة (State Tracking):** يُستخدم متغير صحيح (selectedIndex) كمؤشر رقمي يحدد أي الشاشات هي النشطة حالياً.
- **الاستدعاء الديناميكي (Dynamic Rendering):** يتم تمرير الصفحة المطلوبة لجسم التطبيق (Scaffold.body) عبر الدالة `elementAt(selectedindex)` والتي تجلب العنصر المماثل لرقم المؤشر من القائمة.
- **إعادة الرسم التفاعلية (Reactive UI Updates):** عند النقر على أي تبويب، تُنفَّذ الدالة `onTap` لتعديل قيمة `selectedIndex` وإطلاق `setState()` مما يُعيد بناء الواجهة لعرض الصفحة المحددة فوراً.

Thank you